

Project documentation, methodology, and product requirements planning

Wed, 12 Aug 26

Meeting Purpose

- First meeting with the client (Jan) to align on project documentation, methodology, and product requirements
 - Focus: what is expected in the documentation site, how to follow Agile/Scrum, and what the product must do
-

Client/Product Requirements

- App for local sports team management, focused on football
 - A coach creates and manages their own team through the system
 - No need to track other teams' data, only the coach's own team
 - Focus on one sport first; can expand later if capacity allows
 - Roles to consider: Coach (primary), Assistant/Admin (secondary, to be discussed)
 - Coach is the main role; assistant role capabilities still to be decided
 - Jan will ask about the thought process behind decisions made during development
-

Project Documentation Requirements

- Documentation site is a core deliverable; rubric checks it directly
- Must include:
 - Tech stack description: clear, descriptive explanation of how frontend, backend, and API interact (React, TypeScript, etc.)
 - High-level architecture diagram (colour or black and white, any format)
 - Frontend design structure: folder layout, pages, helper functions, utilities, APIs
 - Backend design structure
 - Database design: tables or ERD diagrams showing relationships
 - Product backlog (as user stories)

- Methodology section (10 marks)
 - UI wireframes or prototype (can be done after implementation; link to Figma/AI design tool acceptable)
 - Roadmap/milestones (can be added after implementation, but must be present by end)
 - Bug/issue tracker (from Sprint 1, to have data ready for Sprint 2 marking)
 - Link to Trello dashboard shared in documentation
 - Share documentation site link with Jan as soon as possible
 - Jan needs access to: documentation site, repo, Trello dashboard, Gitea issue tracker
-

Methodology and Sprint Requirements

- Methodology: Agile, specifically Scrum framework (same as last semester)
- Each sprint must include:
 - Sprint planning session: summary of what was discussed, screenshot/record of the meeting as evidence
 - Sprint backlog: user stories with assignees per feature/requirement
 - Regular standups: every 3 days (agreed by the team)
 - Standup format: voice notes or meetings; progress updates only, no need for excessive detail
 - Sprint review/retrospective: to be scheduled by scrum master
- Scrum master responsibilities:
 - Record meetings, take screenshots, write summaries
 - Set sprint review and retrospective dates
 - Jan will follow up with the scrum master directly on methodology compliance
- Scrum master: team agreed two people is preferable to share responsibility; final decision still pending
- Once scrum master(s) decided, inform Jan immediately

- Standup cadence: every 3 days; Jan may or may not attend but will review updates
 - Daily standups are optional and informal (voice notes acceptable)
-

Functional Requirements

Athlete Management

- Coach can add, view, edit, delete, or archive an athlete
- Athlete profile: details to be decided by the team (name, age, position, etc.)
- Stats page: goals, red cards, yellow cards, season totals, career totals (team to decide scope)
- Stats can live on athlete profile or a separate dedicated page
- Search athletes by name; filter by attributes (e.g., age)
- Archive vs. delete: team to decide logic (e.g., injured player = archive, not delete)
- Jan will ask for the reasoning behind delete/archive decision

Event Management

- Create, edit, cancel, delete, and view events
- Event types: match, training session, team meeting
- View options: list view, card view, or calendar view
- Calendar view: click event to see details, edit from pop-up
- Live match recording:
 - Match timer starts automatically at set start time
 - Dashboard shows live match indicator; click to go to timeline
 - Timeline shows events (goals, yellow cards, red cards) with timestamps
 - Events logged via pop-up: select event type from list, select player from team list (no manual name entry)
 - Logging an event auto-updates that player's statistics

Dashboard

- Shown on login

- Displays: total athletes, total events, upcoming events (first 5 shown, with option to view all)
- Live match shown on dashboard if active; click to open timeline

Statistics

- Athlete statistics: goals, cards, appearances, season/career breakdown
- Team statistics: wins, losses, matches played (scope to be discussed)
- Events can optionally be tagged by season or type (friendly, league, training)

Authentication and Roles

- Authentication is required (login/logout)
- Coach: primary role, creates the team, manages everything
- Assistant/Admin: secondary role, associated with an existing team created by a coach
- Assistant may be able to add athletes and log live match events (to be decided)
- Team must decide: who creates the team first, and how role selection works at registration
- Each team must have a dedicated table in the database; accounts are associated with a team

Non-Functional Requirements

- Code quality: clean codebase structure, to be reflected in frontend design documentation
- Database: PostgreSQL confirmed
- Bug/issue tracker: Gitea issue tracker for bugs; Trello for backlog and sprint management
- Gitea should ideally show graphs or visual data on bug resolution velocity
- If Gitea graphs are insufficient, find another creative visual to show bug resolution progress
- Jan wants to see something visually clear, not just a list of bugs

Decisions Made

- Sport focus: football only (for now)
- Methodology: Agile/Scrum (same framework as last semester)
- Standup cadence: every 3 days
- Bug tracking: Gitea issue tracker
- Sprint/backlog management: Trello
- Scrum master: two people preferred (final names TBD)
- No need to track opposing team data; focus only on the coach's own team
- Sprint 1 should focus on foundational features only; advanced features can be added if time allows

Open Questions / Clarifications Needed

- Who will be the scrum master(s)?
- What specific details will the athlete profile include?
- Will stats live on the athlete profile page or a separate page?
- Delete vs. archive athlete: what is the logical distinction?
- Will the assistant role be able to add athletes and/or log live match events?
- Who creates the team first during registration (coach vs. admin)?
- Will events be associated with a season, or tagged by type (friendly, league, training)?
- Will markers meet with the team, or review documentation independently?
- How many sprints are planned (3 or 4)?
- Confirm with Brendan on any requirements that seem unclear or contradictory

Action Items

- **Decide on and confirm scrum master(s)** Two people preferred; inform Jan once decided so sprint planning responsibilities can be assigned.
- **Conduct sprint planning session and document it** Record the meeting (screenshot or voice note), write a summary, assign user stories in the sprint backlog, and upload everything to the documentation site. Share the link with Jan.

- **Set up and share documentation site** Include tech stack, architecture diagram, frontend/backend/database design, product backlog, methodology section, and roadmap. Share link with Jan as soon as possible.
 - **Share access to Trello dashboard and Gitea issue tracker with Jan** Jan needs visibility into both tools; confirm Gitea can display bug resolution graphs or find an alternative visual.
 - **Finalise and document role/authentication logic** Decide coach vs. assistant capabilities, team creation flow, and account-team association; send Jan a summary document by end of week.
-

Next Steps

- Team to discuss and finalise scrum master(s) and inform Jan
 - Conduct sprint planning; share meeting link with Jan (he may attend to observe)
 - Set up documentation site and begin populating all required sections
 - Confirm standup schedule (every 3 days) and begin standups
 - Finalise product decisions (roles, athlete details, stats scope, delete/archive logic) and send Jan a written summary
 - Confirm any unclear requirements with Brendan; loop Jan in if there is a conflict
-